

MAHARASHTRA IGR DATA PROCESSING PIPELINE

Comprehensive Technical Architecture & Step-by-Step System Documentation (project.py)
Automated Ingestion, Entity Resolution, Spatial Enrichment, & PostgreSQL Loading

1. Executive Summary & Core Mission

project.py serves as the master production orchestrator responsible for transforming raw, semi-structured Inspector General of Registration (IGR) property registration filings into a standardized, validated, and enriched relational dataset. Operating across multiple real estate micro-markets (including Mumbai, Pune, and Thane), the pipeline handles complex challenges including nested JSON extraction, ML-based phonetic and n-gram project name clustering, traditional Marathi land area parsing (Hectare-Are-Guntha), MahaRERA master reconciliation, automated non-RERA sequencing, buyer demographics geo-enrichment, and schema-strict PostgreSQL ingestion.

2. Pipeline Execution Modes

Mode	Scope & Steps	Operational Use Case
Mode [1] Run from START	Steps 1 through 19 (End-to-End)	Processes freshly delivered LLM files from Google Drive (2. LLM Processed Data). Executes full standardization, outputs manual review file, resumes after approval, and uploads to DB.
Mode [2] Run from STEP 7	Steps 7 through 19 (Resume Mode)	Directly loads a verified/corrected manual review workbook from Google Drive (3. Manually Corrected) and executes RERA matching, NR assignment, coordinate enrichment, and upload.
Mode [3] Run STEP 18 ONLY	Steps 18 & 19 (Direct Parquet)	Loads an existing verified final processed Excel dataset, converts it directly to compressed Apache Parquet format, and launches final_code.py for PostgreSQL loading.

3. Detailed Step-by-Step Technical Guide (Steps 1 to 19)

Pre-Step: City Configuration & Google Drive Binding

Initializes the active city context via city_config.py. Dynamically links to local Google Drive desktop shortcut paths (G:\.shortcut-targets-by-id\...) for input, manual review, and final directories. Loads city-specific Saleable-to-Carpet divisors (e.g., 1.45 for Mumbai, 1.35 for Pune from divisor.py).

Step 1: Input File Discovery (Location-Wise Drive Browser)

Recursively scans 2. LLM Processed Data on Google Drive. Parses and displays available localities (e.g., Andheri, Wakad, Kothrud). The operator selects the target location interactively. Preserves location name globally for automated folder structuring downstream.

Step 2: DictToColumn JSON & Dictionary Extraction

Executes DictToColumn.py. Raw IGR entries often contain stringified dictionary representations in metadata columns. This module flattens all nested key-values into distinct, typed DataFrame columns. Automatically verifies city identity.

Step 3: Standardization of Initial Project Name

Creates final_project_name by evaluating project_name_en. If missing or whitespace-only, it falls back to building_name_en and logs audit provenance in final_project_name_status ('Project Name Considered' vs 'Building Name Considered').

Step 4: Static Dictionary Transaction Type Mapping

Imports static.py. Maps localized Marathi deed classifications (Kharedikhat, Bhadepatta, Bakshispatra, etc.) to standardized English legal transaction types (Sale Deed, Lease Deed, Gift Deed, etc.).

Step 5: High-Level Transaction Categorisation

Applies transaction_categorizer.py to categorize records into high-level analytical categories: Sale (capital property transfers), Lease / Mortgage (rental agreements and bank liens), and Other.

Step 5.5: Marathi Revenue Village to Locality Mapping

Queries PostgreSQL public.transactions to map village_name_marathi / areaname to canonical location_name and registered_document_village_name. Ensures zero data loss for historical municipal village boundaries.

Step 6: Hybrid Project Clustering & Marathi Area Conversion

The core machine-learning component via project_name_std_and_area_conversion.py:

- Part A (Entity Resolution): Strips legal suffixes (CHS, Phase, Wing). Generates character n-gram TF-IDF matrices. Calculates pairwise cosine nearest neighbours, checks conservative rejection heuristics, and executes greedy clustering.
- Part B (Marathi Area Parsing): Regex-parses Hectare-Are-Sq.m, Guntha, Are, Sq.ft, and Sq.m into normalized square meters.
- Part C (Unit & Floor): Extracts flat numbers (unit_number) and floor numbers (floor_number).
- Part D (Carpet Derivation): Applies city divisors (1.45 for Mumbai, 1.35 for Pune) to derive net_carpet_area_sq_m for Sale transactions.
- Export: Saves multi-sheet review workbook <Location>_for_manual.xlsx to Google Drive (3. Manually Corrected/<Location>/).
- Notification: Optional automated SMTP email with review instructions, deadline banner, and attachment/link to colleagues.

Step 7: Load Manually Corrected Review File

Pipeline pauses for human review. Once project name flags and ambiguous areas are verified, Step 7 loads the corrected workbook directly from Google Drive or custom path. Step 6 file is loaded instantly on 'Enter'.

Step 8: Standardize Database Column Schema

Executes rename_columns() to align raw fields to production PostgreSQL schema: docno → document_number, consideration_amt → agreement_price, marketvalue → guideline_value, sellerparty → seller_name, purchaserparty → buyer_name, flat_number → unit_number, srocode → sub_registrar_office_code.

Step 9: Categorise Property Types

Normalizes diverse property descriptions into six canonical types: Flat (Apartments, Rooms, Houses), Office (Commercial Offices), Shop (Retail, Restaurants), Villa (Bungalows, Row Houses), Plot (Open Land), and Others.

Step 10: Buyer Location & Postal PIN Code Enrichment

Uses regex to extract 6-digit Indian PIN codes from buyer_name. Merges with postal_pincode.csv to append buyer_locality, buyer_district, and buyer_state.

Step 11: MahaRERA Master Dataset Matching

Runs rera_matching.py against official state MahaRERA datasets (e.g., mumbai RERA GRAND EXCEL VERSION.xlsx). Appends official RERA index, verified project names, geocoded project coordinates (project_latitude, project_longitude), and configurations (BHK).

Intermediate: Financial & Chronological Calculations

- Derives registration quarters (e.g. Q1-2026) and standardizes date formatting to DD/MM/YYYY.
- Computes net_carpet_area_sqft = net_carpet_area_sq_m * 10.7639.
- Computes rate per sq.ft: rate = agreement_price / net_carpet_area_sqft (standardized to column rate).
- Assigns DB defaults: state_name='Maharashtra', data_source='Igr', is_llm_processed='Yes'.

Step 12: Sequential Non-RERA (NR) Index Assignment

Projects that do not match MahaRERA receive a Non-RERA project ID. Connects to PostgreSQL, finds the current max sequence (SELECT MAX(internal_index_id) WHERE internal_index_id ~* '^nr\d+'), and assigns new incremental IDs (nr18421, nr18422, etc.).

Step 13: Fetch Location Coordinates from Database

Queries public.dim_location for location_latitude and location_longitude based on location_name, ensuring locality-level geospatial fallback for every record.

Step 14: Project Coordinates via Google Places API

Optional geocoding module (project_coordinates.py) to retrieve high-precision project coordinates for unmapped developments.

Step 15: Filter & Order Selective Database Columns

Imports DB_SEQUENCE from db_columns.py. Strictly filters and reorders DataFrame columns to match the exact positional schema of public.transactions in PostgreSQL.

Step 16: Title Case Normalization

Applies Python .title() formatting across all text/object fields (names, locations, categories) for visual consistency.

Step 17: Final Dataset Export (Google Drive & Local)

Saves the cleaned master Excel file (<Location>_final_processed.xlsx) into the dedicated Google Drive folder: 4. Final processed file/<Location>/ with interactive validation.

Step 18: High-Performance Parquet Conversion

Launches parquet_conersion.py to convert the Excel file into an optimized Apache Parquet columnar file under converted_feather_parquet/<City>/<City>_db1.parquet with strict type serialization.

Step 19: Trigger Central Database Upload Pipeline

Executes DB1_DB2_Uploading_Pipeline\final_code.py via subprocess in the active virtual environment to ingest the validated Parquet records directly into PostgreSQL.

4. Post-Upload Outlier Detection System (incremental_outlier_update.py)

Following database ingestion, statistical outlier classification is performed on the newly appended records using incremental_outlier_update.py. This prevents corrupting historical medians while providing automated multi-sheet audit summaries:

- Baseline Model: Computes P1/P99 trimmed medians and IQR boundaries for all historical transactions in the city.
- Dynamic Thresholds: Sets lower_limit = median / 4 and upper_limit = median * 4.
- Mumbai Exceptions: Preserves rules for zero area/price deeds (Not Outlier) and sale deeds < ₹1 Lakh.
- Multi-Sheet Excel Report: Saves a 5-sheet workbook in outlier_summaries/ containing Overview, Classification Breakdown, Location Aggregations, Flagged Outliers, and All Processed Rows.
- Database Update: Performs batch update on PostgreSQL columns: rate, is_outlier, and outlier_type.

5. Master File & Dependency Registry

File / Script	Core Functionality & Role
project.py	Master CLI pipeline orchestrator coordinating Steps 1 through 19.
server.py	FastAPI Web server providing real-time SSE progress streaming and web dashboard.
city_config.py	Central configuration for city IDs, Google Drive IDs, RERA master paths, and email templates.
divisor.py	Defines city-specific carpet area divisors (Mumbai: 1.45, Pune: 1.35, Buildup: 1.2).
DictToColumn.py	Extracts stringified JSON dictionaries into structured tabular columns.
transaction_categorizer.py	Rule-based transaction categorizer (Sale, Lease/Mortgage, Other).
project_name_Std_and_area_conversion.py	TF-IDF n-gram clustering, Marathi area converter, unit/floor parser, and review workbook exporter.
rera_matching.py	MahaRERA fuzzy matching engine for project name and coordinate enrichment.
postal_pincode.csv	Master Indian postal PIN code database for buyer locality and district mapping.
db_columns.py	Canonical PostgreSQL column sequence definition (DB_SEQUENCE).
parquet_conversion.py	High-performance Excel-to-Parquet conversion utility with type validation.
incremental_outlier_update.py	Incremental statistical outlier detection and multi-sheet Excel summary generator.